Cloud DevOps Graduation Project

Under Instructor Ibrahim Adel Supervision.

Eman Zaki Mostafa Zaki

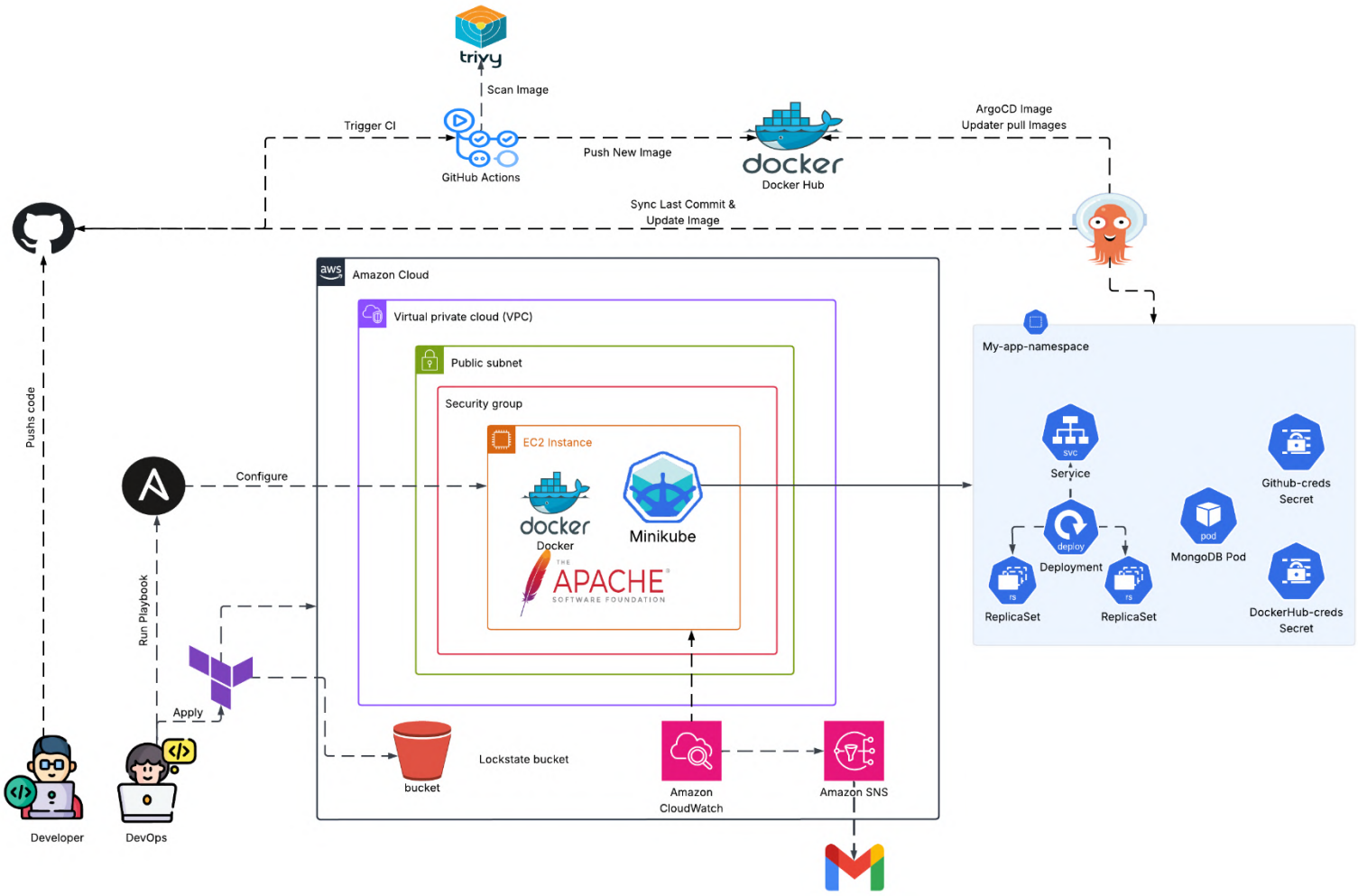
Overview

This project represents the culmination of the DevOps training at the National Telecommunication Institute (NTI), in partnership with iVolve Technologies.

It demonstrates an end-to-end DevOps workflow deployed on AWS ☁️, covering the full lifecycle of modern application delivery. The project includes:

- 🛠️ Infrastructure provisioning using Terraform
- 🐳 Containerization with Docker
- ☸️ Orchestration using Kubernetes
- ⚙️ Configuration management with Ansible
- 🔧 Continuous Integration with GitHub Actions
- 🚀 Continuous Deployment using ArgoCD (GitOps approach)

Project Architecture

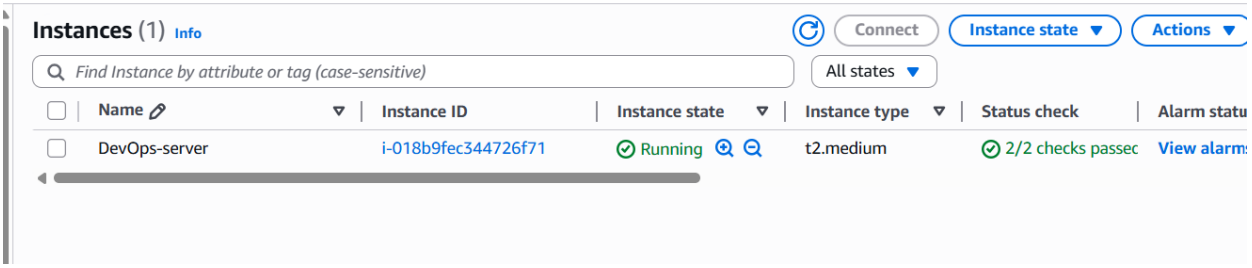


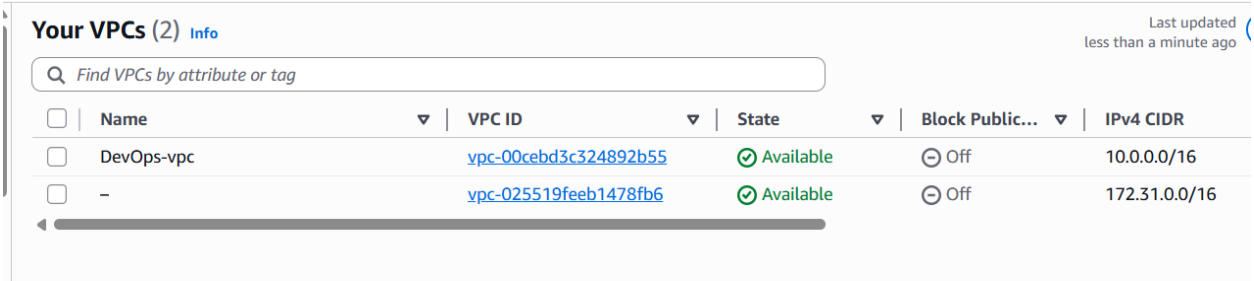
1. Terraform

This project uses Terraform to provision the complete infrastructure on AWS. The following resources are created:

1. 🔐 Key Management
   - tls_private_key and aws_key_pair to generate and register an SSH key for EC2 access.
   - The private key is saved locally in server-info/DevOps-key.pem.
2. ☁️ Networking
   - aws_vpc named DevOps-vpc with CIDR block 10.0.0.0/16.
   - aws_subnet (Public) in us-east-1a, CIDR 10.0.1.0/24.
   - aws_internet_gateway for VPC internet access.
   - aws_route_table and route association for public subnet routing.
   - aws_network_acl to allow all traffic in/out.
   - Outputs: vpc_id, public_subnet_id, igw_id.
3. 🔐 Security
   - aws_security_group named DevOps-sg with:
     - Inbound rules for SSH (22), HTTP (80), and ArgoCD (9000)
     - Outbound rule for all traffic
4. 💻 EC2 Instance
   - aws_instance with:
     - AMI: ami-08a6efd148b1f7504 (Amazon Linux 2)
     - Instance type: t2.medium
     - Public IP assigned
     - SSH key for secure access
     - Tags: Name = DevOps-server
   - Outputs: server_id, server_ip, server_sg_id
   - IP address is stored in server-info/server_ip.txt.
5. 📊 Monitoring and Alerts
   - CloudWatch Alarms:
     - CPU Utilization (> 80%)
     - Memory Utilization (> 80%)
     - EC2 Status Check Failure
   - SNS Topic for alarm notifications:
     - Email alerts sent to: emanzaki774@gmail.com
6. 📝 Local Files
   - Stores:
     - EC2 public IP
     - EC2 private key

```
module.server.aws_sns_topic.alarms: Creating...
module.server.aws_key_pair.server-key: Creation complete after 2s [id=DevOps-key]
module.server.aws_sns_topic.alarms: Creation complete after 4s [id=arn:aws:sns:us-east-1:202533534335:DevOps-server-alarm-topi
module.server.aws_sns_topic_subscription.email: Creating...
module.server.aws_sns_topic_subscription.email: Creation complete after 1s [id=arn:aws:sns:us-east-1:202533534335:DevOps-serve
module.network.aws_vpc.main: Still creating... [00m10s elapsed]
module.network.aws_vpc.main: Creation complete after 16s [id=vpc-001edba99685297ab]
module.network.aws_internet_gateway.igw: Creating...
module.network.aws_subnet.public: Creating...
module.server.aws_security_group.sg: Creating...
module.network.aws_internet_gateway.igw: Creation complete after 2s [id=igw-0dec782d03514a0ad]
module.network.aws_route_table.public: Creating...
module.network.aws_route_table.public: Creation complete after 3s [id=rtb-0eb2b4f128ffae88c]
module.server.aws_security_group.sg: Creation complete after 7s [id=sg-0e1d98ed315f49cd4]
module.network.aws_subnet.public: Still creating... [00m10s elapsed]
module.network.aws_subnet.public: Creation complete after 13s [id=subnet-08270b6a27cd46cc5]
module.network.aws_route_table_association.public: Creating...
module.network.aws_network_acl.public: Creating...
module.server.aws_instance.server: Creating...
module.network.aws_route_table_association.public: Creation complete after 2s [id=rtbassoc-036b95601f0cb96b8]
module.network.aws_network_acl.public: Creation complete after 3s [id=acl-0862875be6b6e592d]
module.server.aws_instance.server: Still creating... [00m10s elapsed]
module.server.aws_instance.server: Still creating... [00m20s elapsed]
module.server.aws_instance.server: Still creating... [00m30s elapsed]
module.server.aws_instance.server: Creation complete after 37s [id=i-0723ad8e1ba35f630]
module.server.aws_cloudwatch_metric_alarm.server_mem: Creating...
module.server.aws_cloudwatch_metric_alarm.server_cpu: Creating...
module.server.aws_cloudwatch_metric_alarm.server_status: Creating...
local_file.server_ip: Creating...
local_file.server_ip: Creation complete after 0s [id=c2d9ab838ea2266de21f5b078b0742d6e8eccc58]
module.server.aws_cloudwatch_metric_alarm.server_cpu: Creation complete after 2s [id=DevOps-server-high-cpu]
module.server.aws_cloudwatch_metric_alarm.server_mem: Creation complete after 2s [id=DevOps-server-high-memory]
module.server.aws_cloudwatch_metric_alarm.server_status: Creation complete after 2s [id=DevOps-server-status-check-faild]

Apply complete! Resources: 17 added, 0 changed, 0 destroyed.
```






```
kira@kira-virtual-machine:~/FortStak/Terraform$ ls ../server-info/
DevOps-key.pem  server_ip.txt
kira@kira-virtual-machine:~/FortStak/Terraform$
```

2. Ansible

🧩 Inventory

- Dynamic Inventory is configured using the aws_ec2 plugin to automatically fetch EC2 instances based on tags, regions, or filters.
- Ensures Ansible targets the correct server without hardcoding IP addresses.

🔧 Playbook Overview

🛠️ server.yml - Server Configuration

This playbook prepares the EC2 instance with the following tasks:

1. Install Docker
2. Install Minikube (Kubernetes for local clusters)
3. Add Docker Hub credentials
    - Required to pull private Docker images
4. Install Argo CD
5. Install ArgoCD Image Updater
6. Add Git credentials
    - Enables ArgoCD Image Updater to push image updates to Git repo

🚀 app.yml - App Deployment

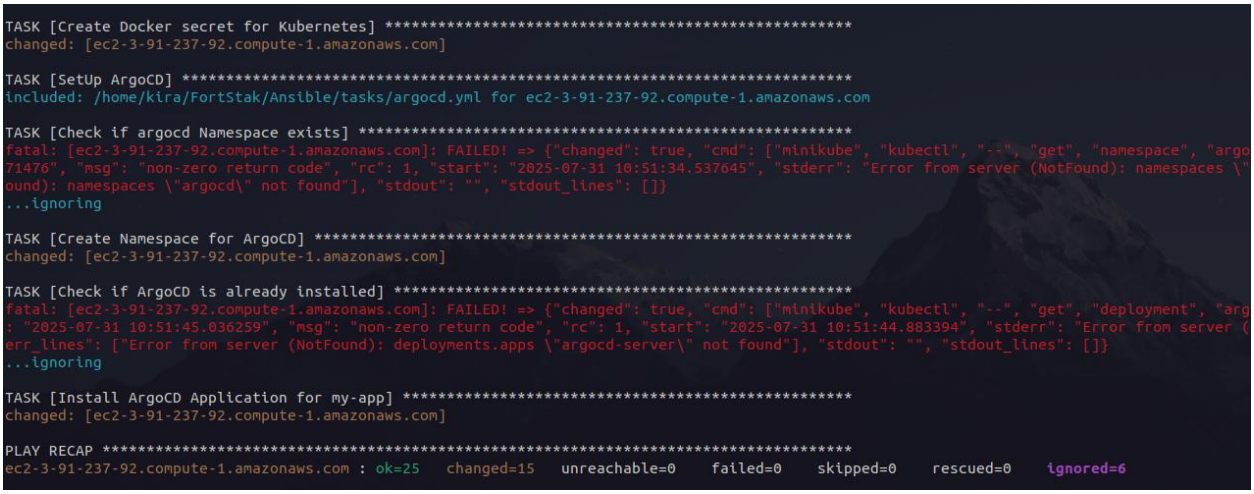
This playbook:

- Deploys your application onto Minikube
- Makes use of ArgoCD to manage the Kubernetes application lifecycle

🔐 Credentials

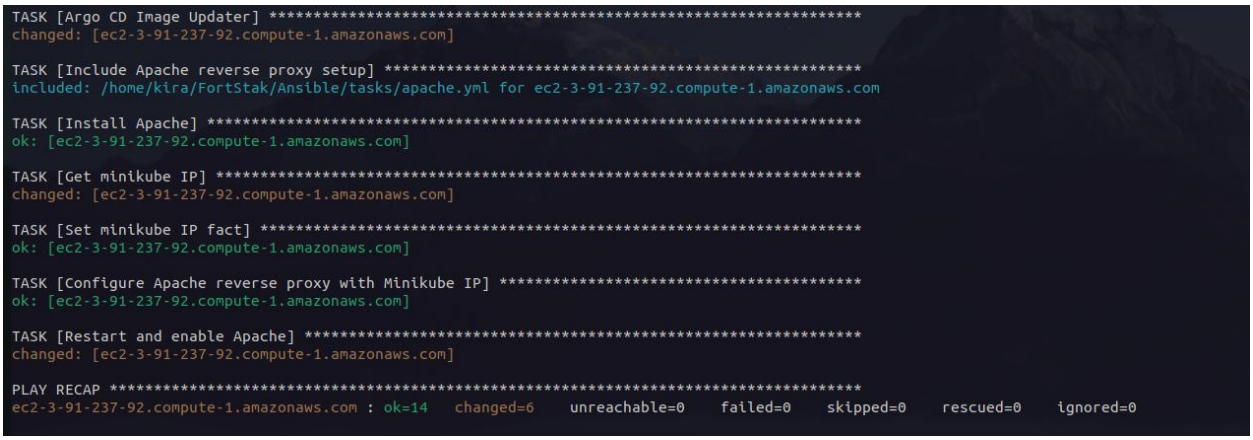
- Docker Hub: Stored securely to allow access to private images.
- Git Credentials: Used by ArgoCD Image Updater to write back updates (e.g., new image tags) to the Git repository.

Server.yml playbook



Note: These are not errors it just to check if the app is installed or not installed it will be ignored.

App.yml playbook



3. Minikube & Kubernetes Deployment

The application is deployed and managed inside a Minikube Kubernetes cluster using Argo CD.

📦 Kubernetes Resources

Within `my-app-namespace`, the following resources are created:

1. Deployment

- Name: `my-app-deployment`
- Replicas: 2
- Containers:
  - Pulls the application image from Docker Hub
  - Uses `imagePullSecrets` for private image access

2. Pod

- Name: `mongodb`
- A separate pod (or Deployment) is used to run a MongoDB database container

3. Secret

- Name: `dockerhub-creds`
- Type: `kubernetes.io/dockerconfigjson`
- Contains Docker Hub credentials to pull private images

🔁 Deployment Flow

1. Minikube is installed and started using Ansible
2. Kubernetes resources (Deployment, MongoDB pod, Secret) are applied
3. Argo CD monitors the Git repository and syncs the state with the cluster
4. Application runs inside `my-app-namespace` with:
   - 2 replicas of the main app
   - MongoDB running as a backend service
   - Docker Hub secret for private image access

4. GitHub Actions

To automate the build, scan, and push process for Docker images.

📁 Workflow File

Located at:
.github/workflows/ci.yml

🧪 Trigger Conditions

The workflow runs automatically on:

- Push to the main branch
- When any of the following files change:
  - Dockerfile
  - Files under Todo-List-nodejs/
  - The workflow file itself: .github/workflows/ci.yml

🛠️ Workflow Jobs

🔹 Job: docker

Runs on: ubuntu-latest

Steps:

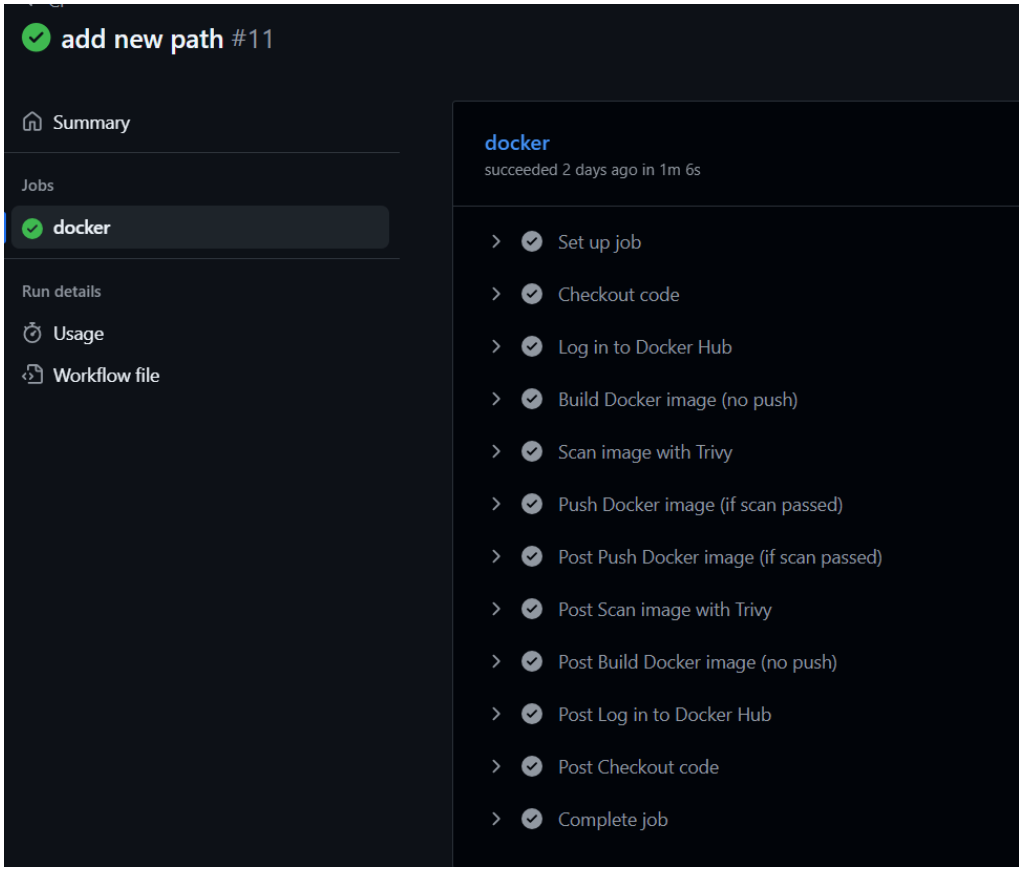
1. Checkout Source Code
   - Uses actions/checkout@v4
2. Authenticate to Docker Hub
   - Uses docker/login-action@v3
   - Requires secrets:
     - DOCKER_USERNAME
     - DOCKER_TOKEN
3. Build Docker Image (No Push)
   - Uses docker/build-push-action@v6
   - Builds the image locally
   - Tags it as:
     ${{ secrets.DOCKER_USERNAME }}/fortstak:${{ github.run_number }}
4. Security Scan with Trivy
   - Uses aquasecurity/trivy-action@0.28.0
   - Scans the built image for vulnerabilities
   - Only reports vulnerabilities with:
     - Severity: HIGH or CRITICAL
     - Types: OS packages and libraries
5. Push Docker Image (Only If Scan Passed)

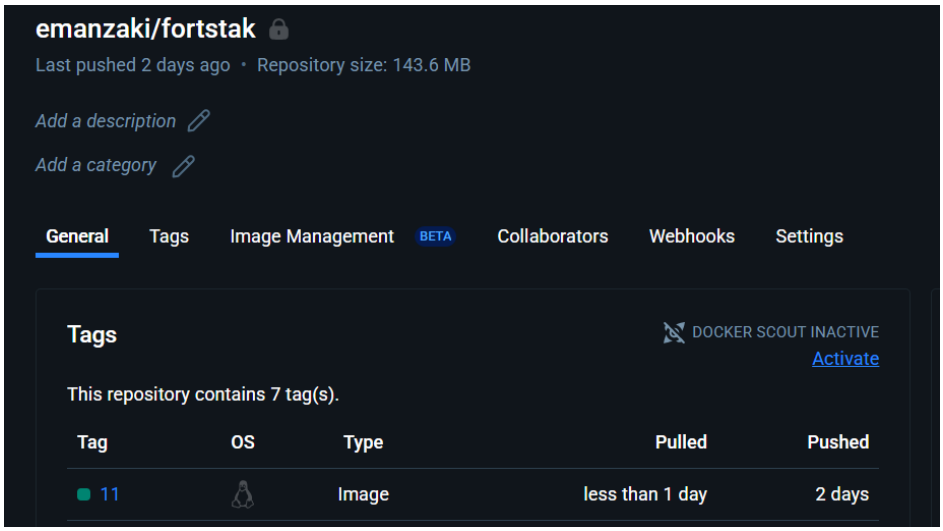
- Pushes the image to Docker Hub
- Reuses the same tag:
  ${{ secrets.DOCKER_USERNAME }}/fortstak:${{ github.run_number }}

🔐 Secrets

secrets in your repository:

- DOCKER_USERNAME: Docker Hub username
- DOCKER_TOKEN: A personal access token from Docker Hub





5. Apache Reverse Proxy Configuration

To expose the application running inside Minikube on port `30080` to the standard web port (`80`), Apache is used as a reverse proxy on the EC2 instance.

🔧 Apache Virtual Host Configuration

Apache forwards all HTTP requests to the application running in Minikube using the following virtual host setup:

```
<VirtualHost *:80>
    ServerName minikube.local

    ProxyPreserveHost On
    ProxyPass / http://{{ minikube_ip_value }}:30080/
    ProxyPassReverse / http://{{ minikube_ip_value }}:30080/

    ErrorLog /var/log/httpd/minikube-error.log
    CustomLog /var/log/httpd/minikube-access.log combined
</VirtualHost>

```

📌 Notes

- {{ minikube_ip_value }} is dynamically replaced with the actual Minikube IP (e.g., `minikube ip` output).
- This allows access to the app via http://<EC2-Public-IP>/
- Logs are stored at:
    - Access logs: /var/log/httpd/minikube-access.log
    - Error logs: /var/log/httpd/minikube-error.log

6. Argo CD & Image Automation

Argo CD for GitOps-based continuous deployment and Argo CD Image Updater for automatic container image updates.

📁 Directory Structure

The Kubernetes manifests are managed under:

```
FortStak/
└── k8s/
    └── dev/
        ├── deployment.yml
        ├── service.yml
        ├── mongodb.yml
        └── kustomization.yaml
```

🔧 Argo CD Configuration

- The Argo CD Application is defined in:

  `.application.yaml`

- Argo CD continuously syncs with the Git repository to apply changes in the `dev` environment.

⚙️ Kustomize

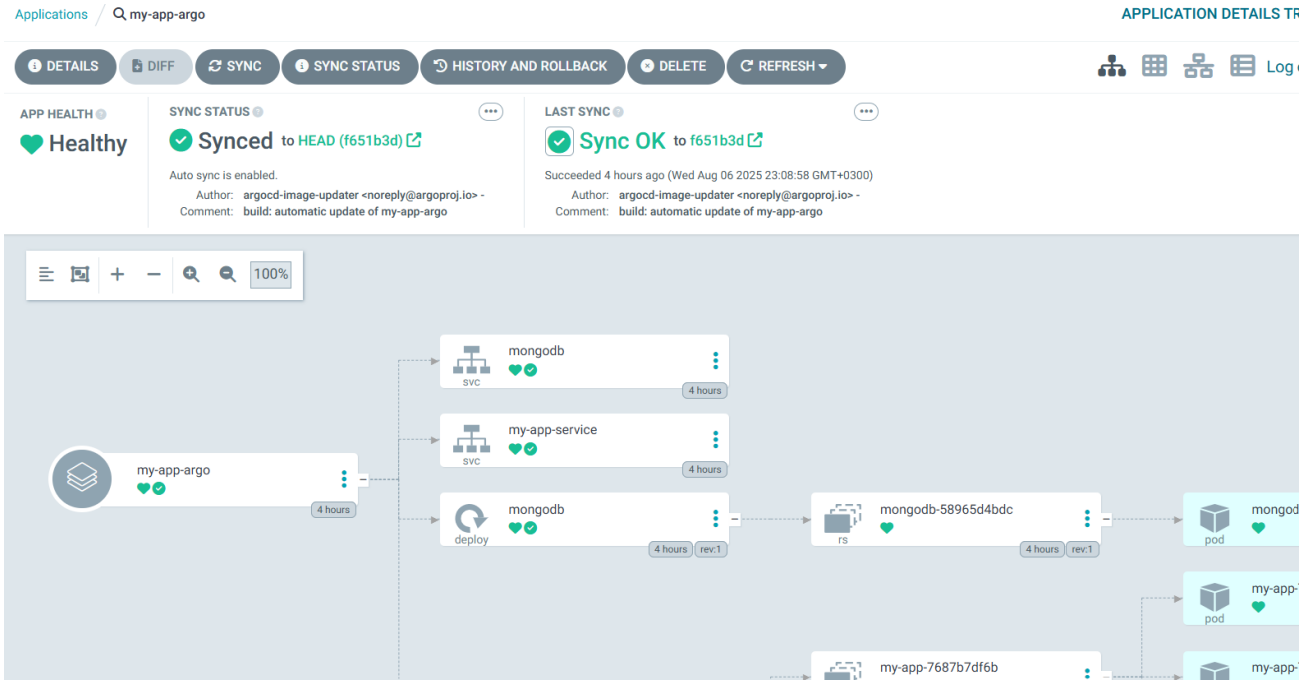
Argo CD uses Kustomize to manage resources.
`kustomization.yaml` includes:

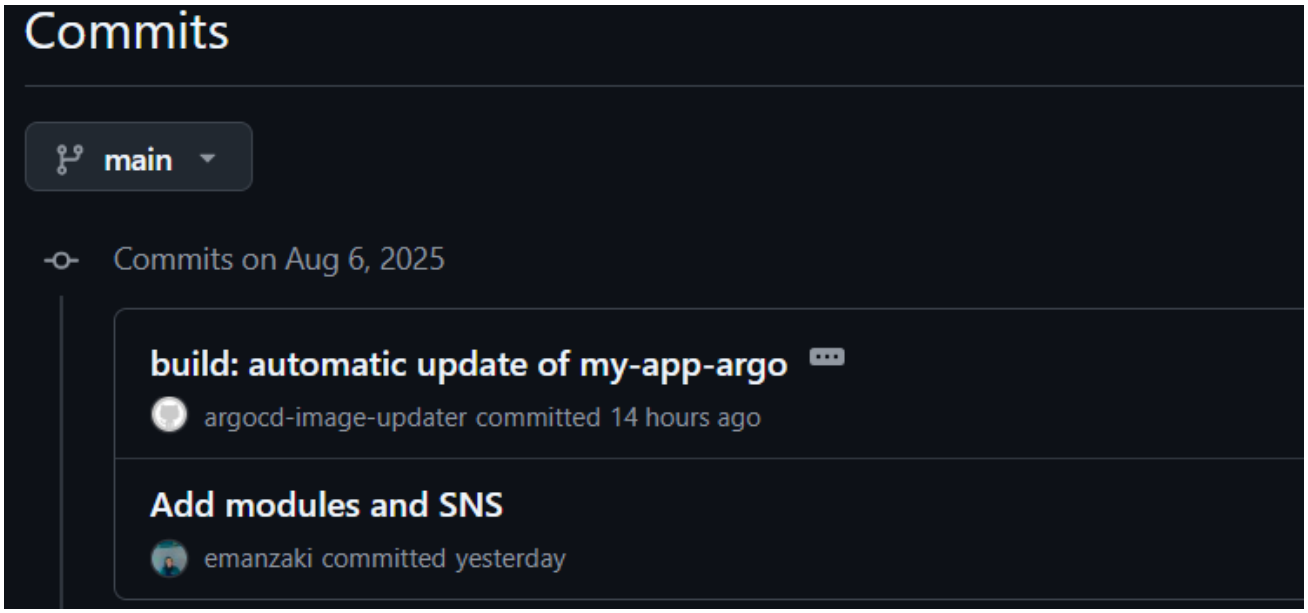
```
resources:
  - deployment.yml
  - service.yml
  - mongodb.yml

images:
  - name: emanzaki/fortstak
```

- This allows Argo CD Image Updater to patch only the image tag without modifying the actual `deployment.yml` file.

Argo CD Image Updater

- Watches the Docker image: emanzaki/fortstak
- Automatically updates the image tag in kustomization.yaml when a new tag is pushed to Docker Hub
- Commits the updated tag back to the Git repository
- The update is applied by Argo CD through GitOps sync





7. CloudWatch Monitoring & SNS Alerts

This project uses AWS CloudWatch and Amazon SNS (Simple Notification Service) to monitor EC2 instances and send email alerts in critical situations.

📁 Terraform Integration

- CloudWatch alarms are provisioned using Terraform.
- SNS topic and email subscription are also created through Terraform code.

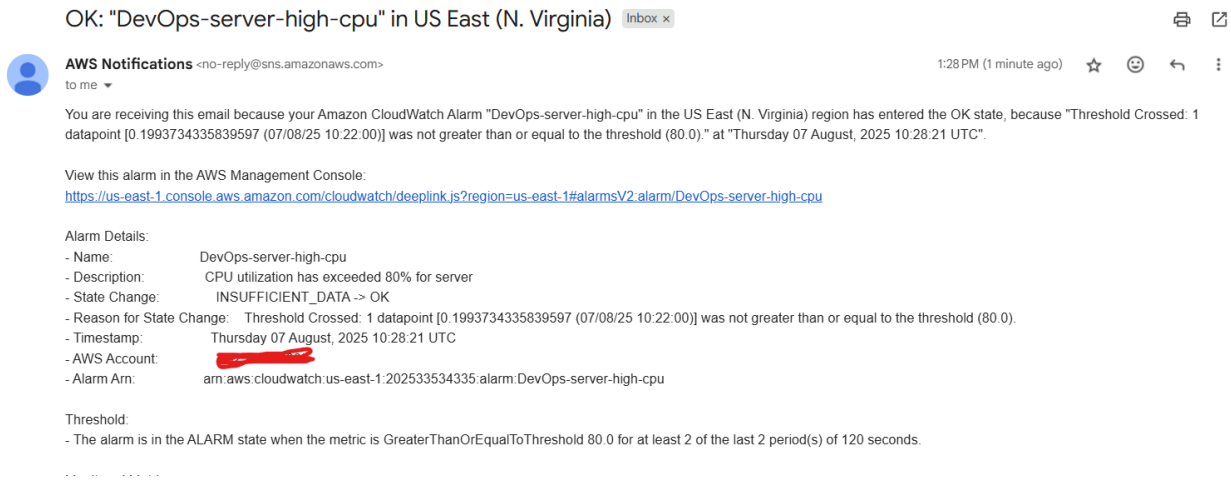
🔔 Alerting Scenarios

Email notifications are sent in the following cases:

- ❌ Instance Status Check Failed
- 🔥 High CPU Usage
- 🛑 Instance Stopped
- 🆘 EC2 Unreachable (InstanceStatusCheckFailed)

📬 Email Notifications

- SNS topic is created and linked to your email address.
- Upon deployment, you'll receive a confirmation email to subscribe to the SNS topic



OK: "DevOps-server-high-cpu" in US East (N. Virginia) Inbox ×

AWS Notifications <no-reply@sns.amazonaws.com> 1:28 PM (1 minute ago)
to me

You are receiving this email because your Amazon CloudWatch Alarm "DevOps-server-high-cpu" in the US East (N. Virginia) region has entered the OK state, because "Threshold Crossed: 1 datapoint [0.1993734335839597 (07/08/25 10:22:00)] was not greater than or equal to the threshold (80.0)." at "Thursday 07 August, 2025 10:28:21 UTC".

View this alarm in the AWS Management Console:
https://us-east-1.console.aws.amazon.com/cloudwatch/deeplink.js?region=us-east-1#alarmsV2:alarm/DevOps-server-high-cpu

Alarm Details:
- Name: DevOps-server-high-cpu
- Description: CPU utilization has exceeded 80% for server
- State Change: INSUFFICIENT_DATA -> OK
- Reason for State Change: Threshold Crossed: 1 datapoint [0.1993734335839597 (07/08/25 10:22:00)] was not greater than or equal to the threshold (80.0).
- Timestamp: Thursday 07 August, 2025 10:28:21 UTC
- AWS Account: [redacted]
- Alarm Arn: arn:aws:cloudwatch:us-east-1:202533534335:alarm:DevOps-server-high-cpu

Threshold:
- The alarm is in the ALARM state when the metric is GreaterThanOrEqualToThreshold 80.0 for at least 2 of the last 2 period(s) of 120 seconds.

GitHub Repo:

https://github.com/emanzaki/FortStak